Modélisation d’un réseau de transport

Avec des classes Java - S2.01/2.02

Jules CARAMELLE, OMAR Mohammad Shafi, HAMI Amine

V1 08/05/2026

Table des matières

1. Version 1 . . . 1
   1.1. Classes . . . 1
   1.2. UML . . . 2
   1.3. Lancement . . . 2
   1.4. Test . . . 2
2. Version 2 . . . 2
   2.1. Classes . . . 3
   2.2. Bugs connus . . . 4
   2.3. UML . . . 4
   2.4. Lancement . . . 4
   2.5. Test . . . 5
3. Version 3 . . . 5
   3.1. Classes . . . 5
   3.2. UML . . . 6
   3.3. Lancement . . . 7
   3.4. Test . . . 7

1. Version 1

Pour la version 1, nous avons décider de créer 5 **classes** et 1 **ENUM**.

1.1. Classes

1.1.1. TypeCout (ENUM)

Nous avons décidés de créer une énumération *TypeCout*, qui contient tous les différents type de coûts que nous pourrons rencontrer.

1.1.2. Voyageur

Cette classe représente l'utilisateur de notre application.
Nous l'avons fait à part pour plus de lisibilité.

Elle regroupe toutes les informations du voyageur : son nom (String), le critère principal qu'il souhaite optimiser (TypeCout), et le moyen de transport qu'il a choisi d'utiliser (ModaliteTransport).

Aussi, si l'utilisateur n'as pas de nom une valur par défaut lui est attribué

1.1.3. CoutConnexion

Cette classe permet de stocker les différents coûts liés à un trajet. Au lieu d'avoir plusieurs variables séparées, nous avons utilisé une **HashMap** qui associe directement un TypeCout (comme le prix, le temps ou le CO2) à sa valeur numérique en Double. Cela permet de regrouper toutes les valeurs au même endroit de façon claire.

1.1.4. Plateforme

C'est la classe principale. Elle gère toutes les données : elle contient la liste de toutes les villes (Lieu) et de tous les trajets (Connexion). C'est elle qui va construire le graphe en filtrant les données selon ce que veux le voyageur. C'est aussi dans cette classe que l'on utilise l'algorithme pour calculer et afficher les meilleurs itinéraires.

1.1.5. Route

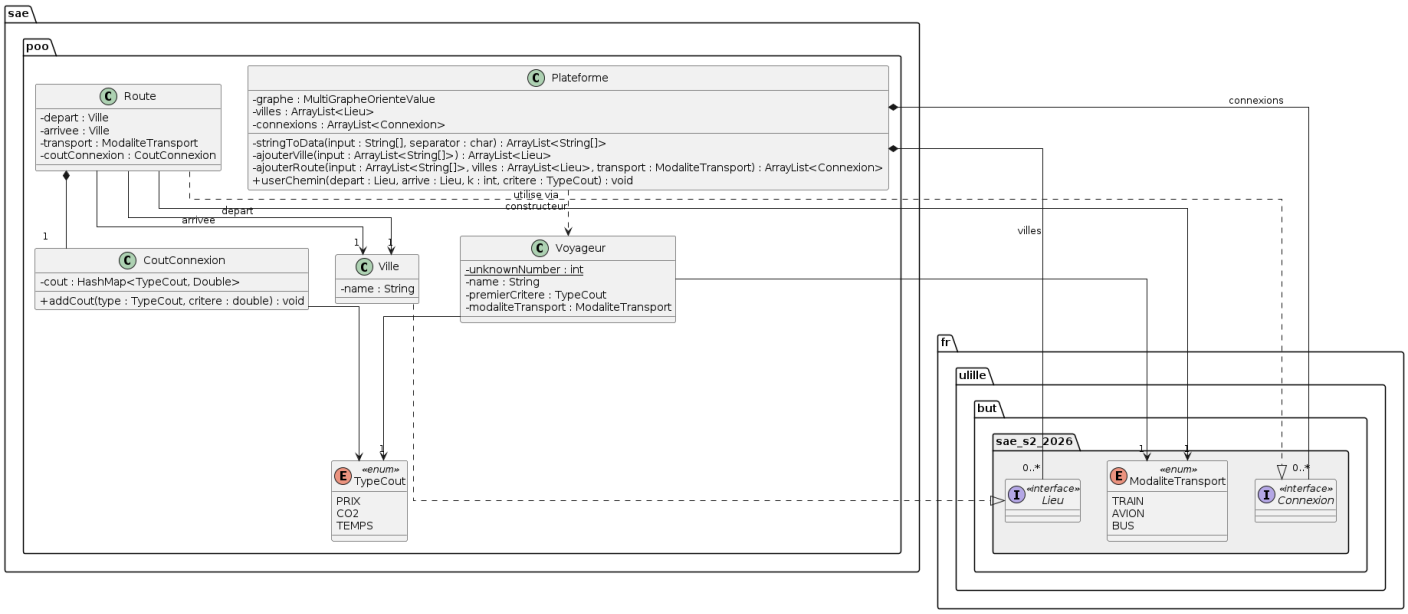
Cette classe représente un trajet entre deux villes. Elle implémente l'interface Connexion qui est exigée par le cahier des charges. Elle relie une ville de départ et une ville d'arrivée, définit le moyen de transport utilisé (ModaliteTransport), et possède un objet CoutConnexion qui contient sa durée, son prix et son bilan carbone.

1.1.6. Ville

C'est une classe très simple qui représente un point d'arrêt sur notre réseau. Elle implémente l'interface Lieu qui est obligatoire pour le projet. Elle stocke simplement le nom de la ville sous forme de texte (String). Nous avons également ajouté des méthodes de Lieu qui sont implémentées.

© Université de Lille — 2026

1.2. UML



1.3. Lancement

Pour complier l'application, il faut d'abord avoir java en version 17 et utiliser la commande suivante :

```
javac -d bin -cp "lib/*" src/sae/poo/*.java
```

Et pour lancer :

```
java -cp "bin:lib/*" sae.poo.Plateforme
```

Notre classe **Plateforme** dans */src/sae/poo* possède un main pour pouvoir tester le fonctionnement, mais surtout comprendre comment utiliser la classe.

1.4. Test

Des tests sont disponibles dans le dossier *test*, ils testent ici un cas d'usage typique de notre application.

Ils sont basé sur **JUnit Jupiter**, dont la librairie et dans le dossier *lib*.

2. Version 2

Lors de la Version 1, nous avions créé plusieurs classes et énumérations permettant d'implémenter les fonctionnalités suivantes :

- Modélisation des données
- Construction du graphe (uni-modale)
- Calcul et présentation d'itinéraires

Pour la Version 2 nous avons respecté les contraintes suivantes :

- Construction d'un graphe multimodale
- Gestion des correspondances
- Présentation épurée
- Implémentation d'exceptions

2.1. Classes

Voici les nouvelles classes :

2.1.1. Exceptions

EmptyCSV : Erreur en cas de fichier source CSV vide.

InvalidCSVData : Erreur en cas de fichier CSV invalide, peut donner le nombre d'arguments en trop.

UnknownCity : Erreur si une ville demandée par l'utilisateur n'est pas dans le graphe.

NoPathFound : Erreur en cas de chemin introuvable entre deux villes

2.1.2. Nouvelles classes

Station : La classe station implémente Lieu et représente une ville est une modalité de transport

Correspondance : La classe correspondance implémente Connexion et représente un lien de correspondance entre deux stations d'une même ville

LiaisonVirtuelle : La classe liaison virtuelle implémente Connexion et représente une connexion à sens unique pour la ville de début vers ses stations et des stations d'arrivée vers la ville d'arrivée.

2.1.3. Classes modifiées

Route : Modifié pour passer d'un lien entre deux villes à un lien entre deux stations.

Voyageur : Suppression de la modalité de transport unique de l'utilisateur, et allègement du constructeur par conséquent.

Plateforme : Dans la plateforme, nous sommes passé d'un graphe construit avec une liste de tableaux de String à deux fichiers csv, un pour les stations et un pour les correspondances. Ensuite plateforme contient désormais la gestion des erreurs qui sont throw. Enfin, la logique complète de la classe a été revue pour passer d'un algo qui gère un graphe avec un seul type de transport, à un algo qui peut gérer des correspondances et plusieurs types de transports.

Pour la gestion du graphe, un problème rencontré a été que l'utilisateur doit pouvoir demander une ville et non une station, il faut donc des super-points qui pour une ville représentent le départ/arrivé, et qui sont liés à toutes les stations de la ville avec un cout nul. Mais si ses points sont présents avec des connexions symétriques sur toutes les villes, l'algorithme KPCC n'empruntera pas les correspondances payant au profit des super points. Donc avant de procéder au KPCC, nous créons le super point de ville de

© Université de Lille — 2026

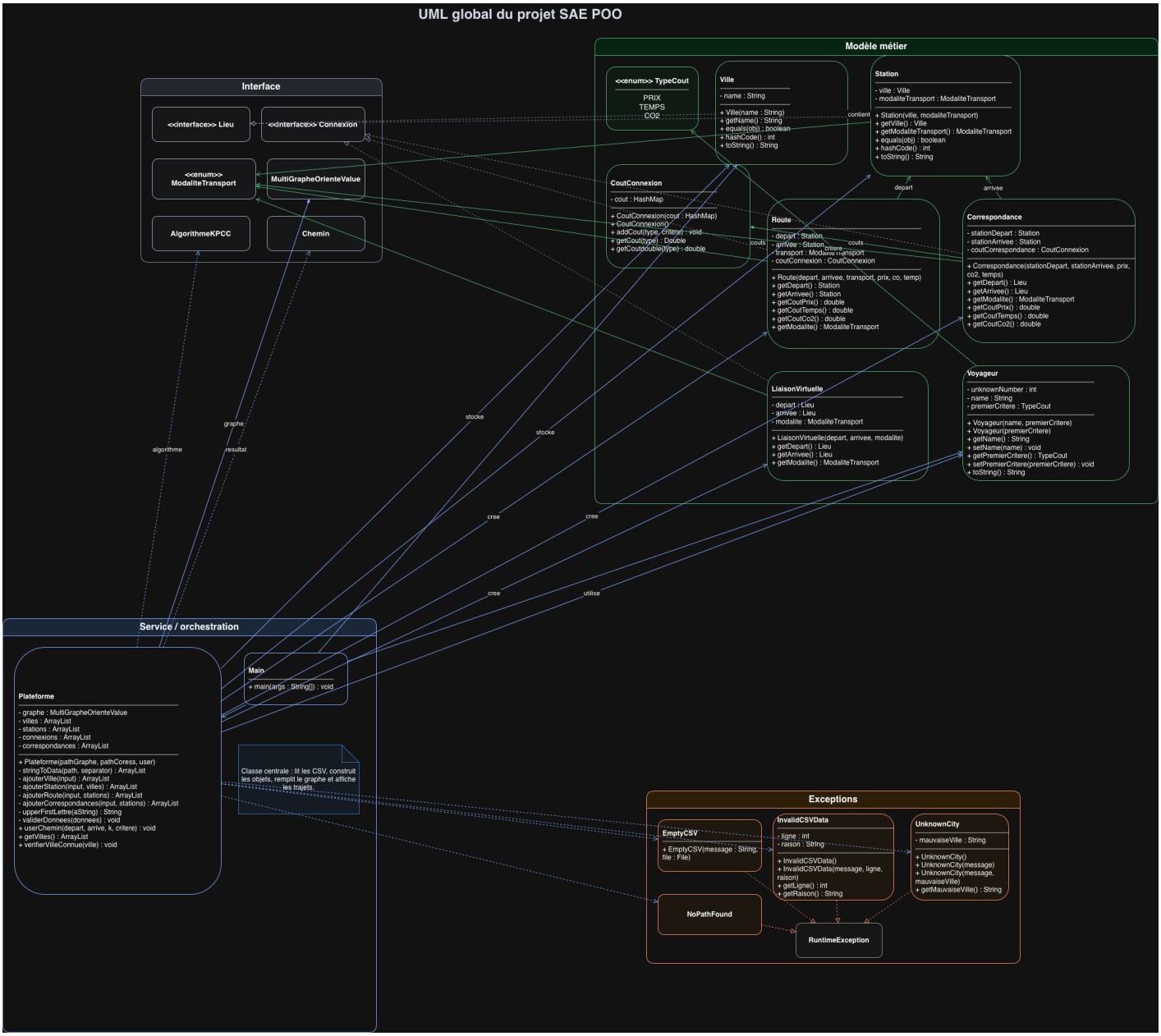
dpéart qui ne va que **vers** les stations de ladite ville et pour l'arrivée ce sont des stations **vers** la ville d'arrivée.

Main : Le main a été changé pour prendre en compte les changement ci-dessus

2.2. Bugs connus

À cause de l'approche vu ci-dessus, il arrive que même si l'utilisateur est arrivé dans une station de la ville d'arrivée, l'algo fait des correspondances dans la même ville avant d'aller vers le super point de la ville.

2.3. UML



2.4. Lancement

Pour complier l'application, il faut d'abord avoir java en version 17 et utiliser la commande suivante :

```
javac -d bin/ -cp "lib/*" src/sae/poo/*.java
```

Et pour lancer :

```
java -cp "bin:lib/*" sae.poo.Main
```

Notre classe **Plateforme** dans */src/sae/poo* possède un main pour pouvoir tester le fonctionnement, mais surtout comprendre comment utiliser la classe.

2.5. Test

Des tests sont disponibles dans le dossier *test*, ils testent ici un cas d'usage typique de notre application.

Ils sont basé sur **JUnit Jupiter**, dont la librairie et dans le dossier *lib*.

Pour les lancer éxecuter les commandes suivantes :

```
javac -cp "lib/*" -d bin/production/A2 src/sae/poo/*.java test/sae/poo/MainTest.java

java -jar lib/junit-platform-console-standalone-1.13.0-M3.jar \
  -cp lib/sae_s2_2026.jar:lib/jgrapht-core-1.5.1.jar:lib/jheaps-
0.14.jar:bin/production/A2:bin/test \
  -c sae.poo.MainTest
```

3. Version 3

Lors de la Version 3, nous avions mis en place un algorithme de recherche de chemins multimodaux avec gestion des correspondances en ligne de commande.

Pour la Version 3, l'application a subi une refonte architecturale majeure pour répondre aux objectifs suivants :

- **Création d'une Interface Graphique (IHM)**.
- **Découplage** de la logique métier et de l'affichage (architecture de type MVC).
- **Persistance des données** (sauvegarde et chargement de l'historique des utilisateurs).
- **Recherche multicritère avancée** avec système de pondération et de contraintes maximales.

3.1. Classes

L'intégration de l'IHM et des nouvelles fonctionnalités de sauvegarde a nécessité la création et la modification de plusieurs classes fondamentales.

© Université de Lille — 2026

3.1.1. Nouvelles classes

VoyageService : C'est la nouvelle classe centrale d'orchestration. Elle agit comme une API interne entre la vue et le modèle (`Plateforme`). Elle isole la logique de recherche complexe (comme `rechercherDetaille()`) et retourne des objets simples prêts à être affichés par l'interface.

ResultatTrajet : Classe interne à `VoyageService`. C'est un conteneur de données (prix, temps, CO2, étapes) utilisé spécifiquement pour transporter les résultats de l'algorithme vers les contrôleurs de l'interface graphique.

Voyage et **HistoriqueVoyage** : Ces classes implémentent l'interface `Serializable`. Elles permettent d'enregistrer les détails d'un trajet sélectionné par l'utilisateur et de stocker une liste complète de ses voyages. L'historique offre également des méthodes d'analyse, comme `evolutionCout()`.

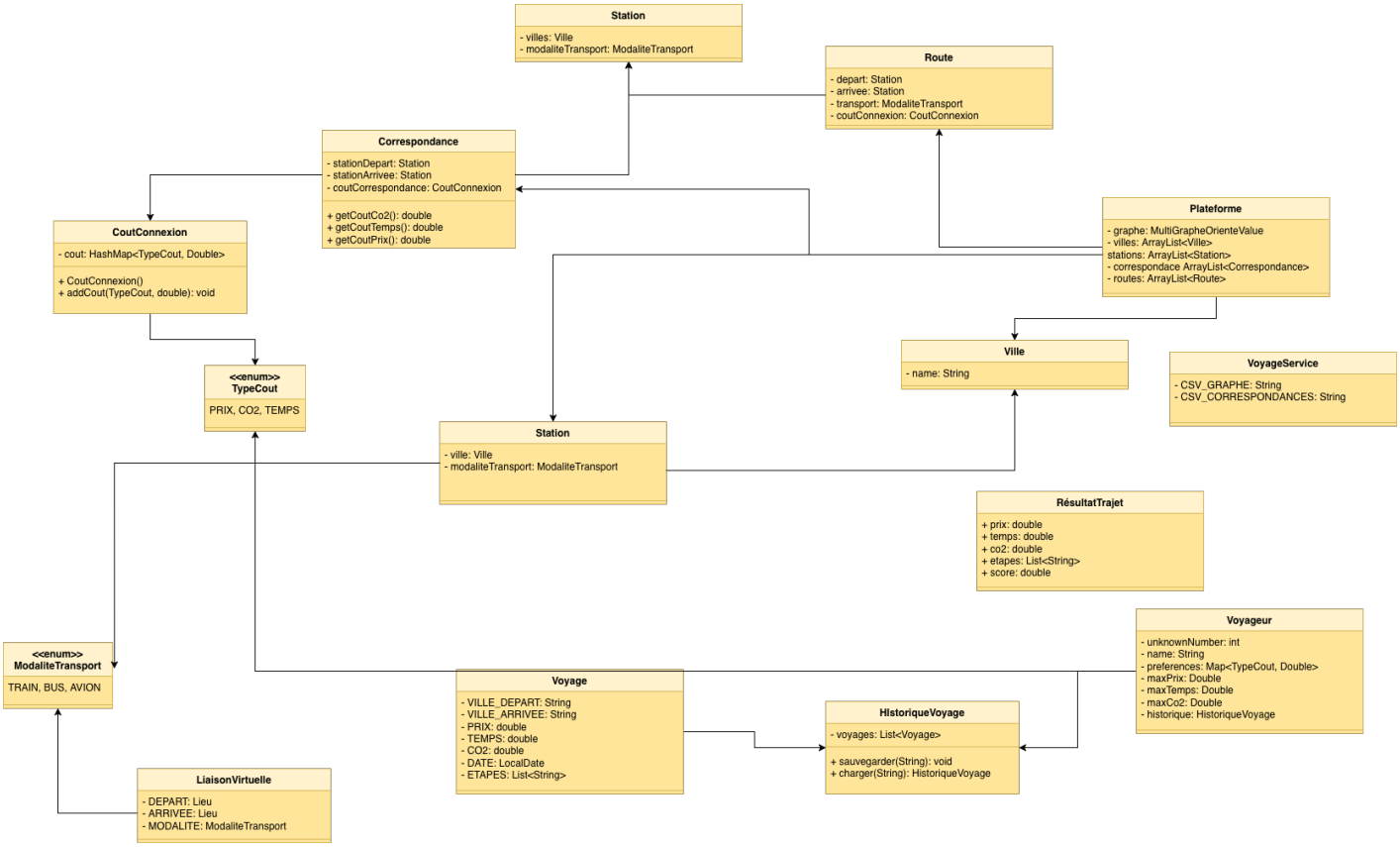
Les Contrôleurs (AccueilController, RechercheController, DetailsVoyageController, etc.) : Nouvelles classes dédiées à la gestion de l'interface. Elles interceptent les actions de l'utilisateur (clics, saisies textuelles) depuis les vues, font appel au `VoyageService`, et mettent à jour l'affichage dynamique.

3.1.2. Classes modifiées

Voyageur : Cette classe a été profondément enrichie. Au lieu d'un unique critère de recherche, le voyageur possède désormais une `Map<TypeCout, Double>` permettant de pondérer ses préférences (Prix, Temps, CO2). De plus, il intègre des contraintes dures (`maxPrix`, `maxTemps`, `maxCo2`) et un objet `HistoriqueVoyage` pour mémoriser sa session.

Plateforme : Ajout de la méthode `respecteContraintes()`. Lors de la recherche de chemins (`calculerChemins`), la plateforme vérifie désormais si les trajets trouvés par l'algorithme KPCC dépassent les limites fixées par le `Voyageur` avant de les valider.

3.2. UML



3.3. Lancement

Pour compiler l'application avec les dépendances:

```
javac -d bin/ -cp "lib/*" src/sae/**/*.java
```

Et pour lancer l'interface graphique :

```
java -cp "bin:lib/*" sae.ihm.Accueil
```

3.4. Test

Les tests ont été étendus pour couvrir les nouvelles fonctionnalités, notamment la sérialisation (sauvegarde de l'historique) et le filtrage multicritères.

Ils restent basés sur **JUnit Jupiter** et peuvent être exécutés avec les mêmes commandes que dans la version précédente :

```
javac -cp "lib/*" -d bin/production/A3 src/sae/poo/*.java src/sae/service/*.java test
/sae/**/*.java

java -jar lib/junit-platform-console-standalone-1.13.0-M3.jar \
  -cp lib/sae_s2_2026.jar:lib/jgrapht-core-1.5.1.jar:lib/jheaps-
0.14.jar:bin/production/A3:bin/test \
```


© Université de Lille — 2026

```
  -c sae.service.VoyageServiceTest
```

© Université de Lille — 2026